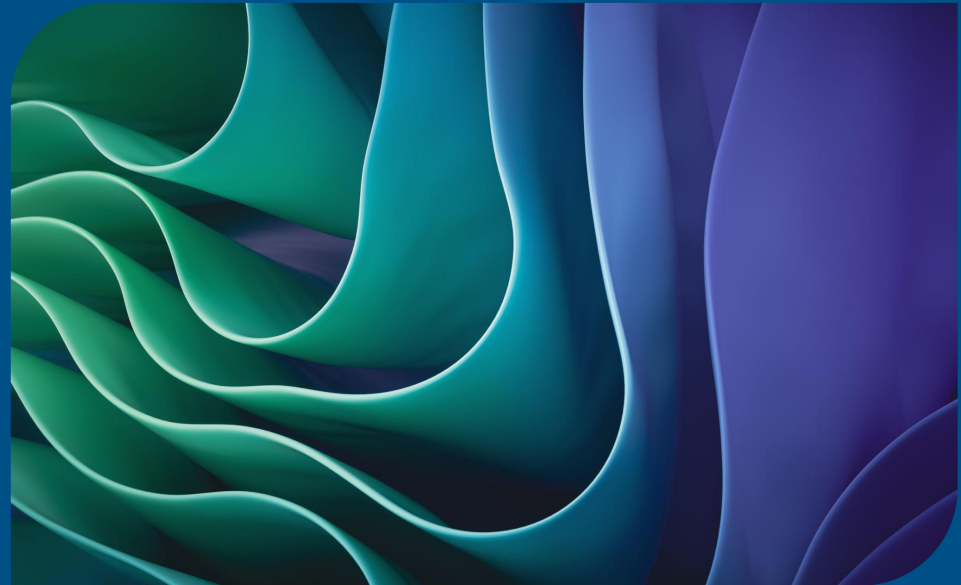


Exploiting Rigid-Body Symmetry in Flow-Matching Motion Planning

Andrea Emir Sevinçel



1. **3D Benchmark**
2. **Baseline**
3. **Explore Canonicalization**
4. **Theoretical Validation**
5. **Discussions & Conclusions**

Flow Matching

Flow Matching is a generative modeling algorithm, that has a model simulate a vector field, such that we give as input some noisy data probability distribution, and as output we get the probability distribution we desire.

Why a distribution? Instead of predicting a single deterministic path, it is better if we learn a conditional generative policy.

We model trajectory generation as a continuous-time flow

BEFORE



AFTER



But in 3 dimensions..

There's one spin you can't get rid of

You can line up start and goal. But the room can still be rolled around that line like meat on a rotisserie, and there is no consistent way to pick which way is "up".

1. **3D Benchmark**
2. **Baseline**
3. **Explore Canonicalization**
4. **Theoretical Validation**
5. **Discussions & Conclusions**

	collision-free	s/query
straight line	15.6%	—
RRT-Connect	99.6%	0.284
expert (RRT+CHOMP)	100%	0.342
learned planner	see <i>next slides</i>	0.067

1. 3D Benchmark
2. **Baseline**
3. Explore Canonicalization
4. Theoretical Validation
5. Drawing a Conclusion

15% → 51%, from a coordinate change

Before

14.6

Straight line, eyes
shut

15.6

After

51.1

Built an Expert Planner

RRT-Connect, random shortcutting,
uniform resampling, and CHOMP
refinement

1. 3D Benchmark
2. Baseline
3. **Explore Canonicalization**
4. Theoretical Validation
5. Discussions & Conclusions

Equivariance can be put into the data (roll augmentation), the operator (frame averaging at inference), or the weights (equivariant architecture).

mechanism	gain
roll augmentation	+0.8
frame averaging ($K=1 \rightarrow 3$)	$+0.68 \pm 0.13$
equivariant architecture	<i>next slide</i>

- Instead of *hoping* it learns the symmetry, I rebuilt it so spinning the room **automatically** spins the answer
- It's a guarantee, not a hope — true even before any training at all
- Same final score. But it gets there **4× faster**

AFTER...	NEW NET	OLD NET
10 rounds	35 . 8	22 . 2
40 rounds	44 . 6	37 . 9
300 rounds	50 . 9	50 . 8

On a whim I told the model, at every point along the path, **how close the nearest obstacle is and which way it is**. It already had that information in its input — it just wasn't using it.

	WITHOUT	WITH
Before the trick	14.6	running...
After the trick	50.8	79.2
+ the new net	50.9	82.2

+28 points. Bigger than everything else I did this summer, combined.

And the fancy network, worth **+0.1** on its own, became worth **+3** once it had something to work with.

Still open. One box is still training. If it comes back high, part of my earlier story needs rewriting — and I'd rather find that out myself.

1. 3D Benchmark
2. Baseline
3. Explore Canonicalization
4. **Theoretical Validation**
5. Discussions & Conclusions

:)

1. 3D Benchmark
2. Baseline
3. Explore Canonicalization
4. Theoretical Validation
5. **Discussions & Conclusions**

Drawing a Conclusion



The best idea of the summer took one afternoon and small compute power. The thing I spent six weeks on was worth a tenth of a point